

High-performance Intrusion Detection System using eBPF with Machine Learning algorithms

NEMALIKANTI ANAND (✉ mail2anand@uohyd.ac.in)

University of Hyderabad

SAIFULLA M A

University of Hyderabad

Pavan Kumar Aakula

University of Hyderabad

Research Article

Keywords: DoS, DDOS, eBPF, Random Forest, Decision Tree, SVM ,TwinSVM

Posted Date: July 6th, 2023

DOI: <https://doi.org/10.21203/rs.3.rs-3140072/v1>

License: © ⓘ This work is licensed under a Creative Commons Attribution 4.0 International License.

[Read Full License](#)

Additional Declarations: No competing interests reported.

High-performance Intrusion Detection System using eBPF with Machine Learning algorithms

Nemalickanti Anand^{1*}, Saifulla M A^{1†} and Pavan Kumar Aakula^{1†}

^{1,1*}SCIS, UoH, Gachibowli, Hyd, 500046, TS, India.

*Corresponding author(s). E-mail(s): mail2anand@uohyd.ac.in;

Contributing authors: saifullah@uohyd.ac.in;

pavankumaraakula1998@gmail.com;

[†]These authors contributed equally to this work.

Abstract

Denial of Service (DoS) and Distributed DoS (DDoS) attacks are standard problems organizations that rely on network services face. Detecting these attacks promptly and accurately is crucial to mitigating the damage caused. This paper proposes an Intrusion Detection System (IDS) that utilizes the extended Berkeley Packet Filter (eBPF) with machine learning algorithms, namely Decision Tree (DT), Random Forest (RF), Support Vector Machine (SVM), and TwinSVM. eBPF is a bytecode-based virtual machine that runs programs without modifying the kernel source code. It can implement various services, such as observability, security, and networking. Socket filters are an eBPF program attached to the socket in the Linux kernel that allows for efficient filtering and manipulation of network packets at the socket after packets are received from the network stack. Packets that are filtered at the socket level before entering the user space. The steps involved in the proposed model are: a) collecting data from famous repository, CIC-IDS-2017. b) Once the raw data is obtained, it undergoes preprocessing, which includes data transmission, cleaning, reduction, and discretization. c) Following the preprocessing step, an ANOVA F-test extracts specific features from the preprocessed data. d) Lastly, the extracted features are analyzed for intrusion detection using various ML algorithms: DT, RF, SVM, and TwinSVM. e) The eBPF program captures network traffic and utilizes trained model parameters to detect attacks within the kernel. Our experimental results show that the accuracy of our proposed ML algorithms, DT, RF, SVM, and TwinSVM, outperforms the existing related work: 99.38, 99.44, 88.73, and 93.82, respectively. The experimental code available in <https://github.com/NemalickantiAnand/Project.git>

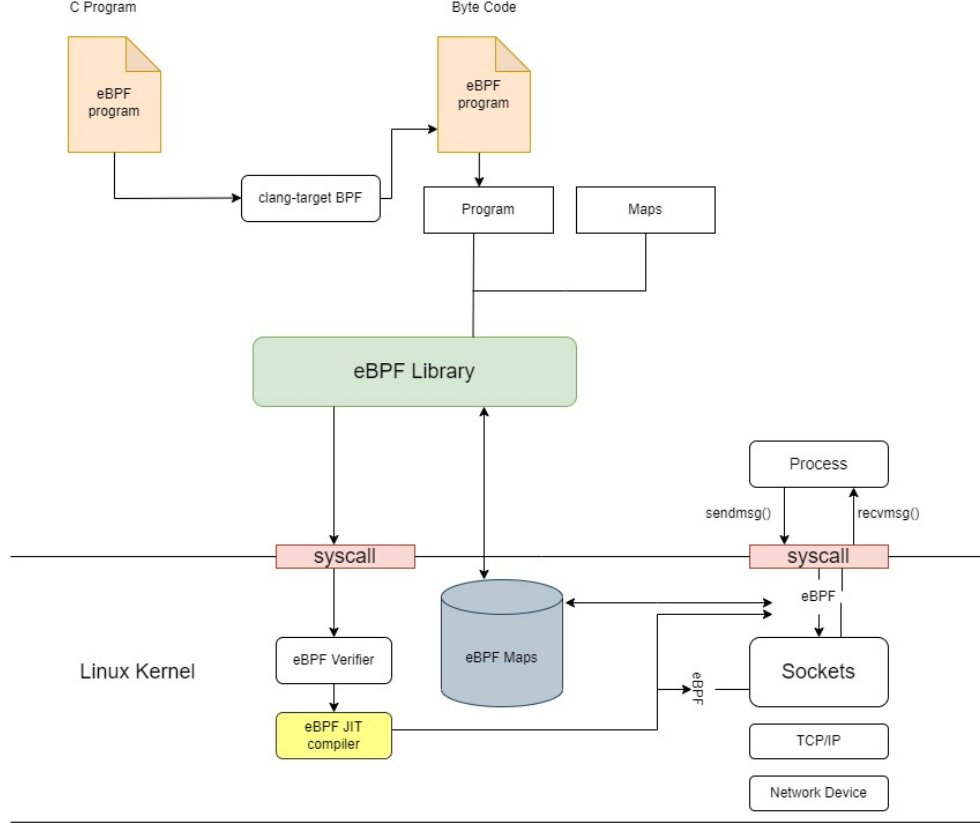


Fig. 1: eBPF Architecture

Keywords: DoS, DDOS, eBPF, Random Forest, Decision Tree, SVM ,TwinSVM

1 Introduction

eBPF stands for extended Berkeley Packet Filter. From that name, we can see it is a packet filter. However, it is now used for performance monitoring, tracing, and optimization. Users are given the ability to build real-time programmes that interact with the Linux kernel and the various components of the system. In the past, practically all of the content on web sites was written in a format known as hypertext markup language (HTML). The act of browsing websites has evolved into a full-fledged application, and web-based technology has mostly supplanted traditional software. This development was made possible by programmability, which was made possible with the advent of JavaScript [1][2][3].

In the same vein, eBPF is the solution to use if we want to dynamically update the kernel. This solution is analogous to the one that JavaScript provides for HTML. The Linux kernel is undergoing a transformation brought on by eBPF, similar to how JavaScript altered the web. Users are able to execute programmes in a secure setting by using eBPF, which permits the execution of sandboxed programmes inside the context of privileged operations within the operating system[4]. Since the programmes are run in the kernel, this results in a much reduced amount of overhead compared to using native kernel modules. eBPF enables a wide variety of application cases, ranging from simple network monitoring to intricate performance optimisation and security checks [1][2][3]. eBPF programmes are event-driven programmes that are activated when a hook point is passed by the kernel or an application. The terms system calls, “function entry and exit,” kernel tracepoints,” and network events” are all examples of pre-defined hook points. If there is not already a pre-defined hook, the kernel probe (probe) and the user probe will be able to attach eBPF programmes practically anywhere in the user applications or the kernel itself [1][2][3][4]. The eBPF architecture, as seen in the Figure 1. eBPF is made up of a number of different components, such as a virtual machine, a collection of libraries, and a set of maps, when seen from the viewpoint of its underlying architecture. While the eBPF programme is being run via the virtual machine, the libraries are providing a set of helper functions[1][2][3][4] for the eBPF programme to make use of.

eBPF is a framework that gives users the ability to load and execute their own customised programmes directly into the kernel of the operating system. When an eBPF programme is loaded into the kernel, a verifier checks to see whether it is safe to execute and decides whether or not to reject it if it is not safe. After it has been loaded, an eBPF programme must be connected to an event in order for it to be activated whenever the associated event takes place. With the help of the Low-Level Virtual Machine (LLVM) compiler[5], we are able to transform pseudo-C code into eBPF bytecode. This is necessary since the Linux kernel anticipates that eBPF programmes will be loaded as bytecode. It is possible to load an eBPF programme into the Linux kernel by using the BPF system call. This is commonly accomplished by utilising one of the eBPF libraries that are currently available. Before the programme can be connected to the specified hook, it must first be loaded into the Linux kernel and then go through the following two steps: Both the verification step and the Just-in-Time (JIT) [6][1][2][3][4] compilation step ensure that the eBPF programme is safe to run. The JIT compilation step optimises the execution speed of the programme by translating the generic bytecode of the programme into the machine-specific instruction set. This allows eBPF programmes to run as efficiently as natively compiled kernel code or as code loaded as a kernel module [1][2][3][4].

eBPF programmes have the capability to save and communicate information that they have gathered with one another. For this reason, eBPF programmes may make use of the idea of eBPF maps, which are comparable to arrays or hash tables and enable eBPF programmes to store and retrieve data in real time. eBPF maps[1][2][3][4][4] are similar to arrays and hash tables. Through the use of a system call, eBPF applications and programmes running in user space are able to get access to eBPF maps. The following kinds of maps are supported in this list: eBPF programmes are composable

Table 1: Overall dataset description of CIC-IDS-2017

Number	Classes	Number of Records	Total Data
1	“BENIGN”	2273097	80.30%
2	“DoS Hulk”	231073	8.16%
3	“DDoS”	128027	4.52%
4	“DoS GoldenEye”	10293	0.36%
5	“DoS slowloris”	5796	0.20%
6	“DoS Slowhttptest”	5499	0.19%
7	“PortScan”	158930	5.61%
8	“FTP-Patator”	7938	0.28%
9	“SSH-Patator”	5897	0.21%
10	“Bot”	1966	0.06%
11	“Web Attack-Brute Force”	1507	0.05%
12	“Web Attack-XSS”	652	0.02%
13	“Infiltration”	36	0.001%
14	“Web Attack-Sql Injection”	21	0.0007%
15	“Heartbleed”	11	0.0003%

using the ideas of tail and function calls, allowing for the creation of hash tables, arrays, Least Recently Used (LRU), ring buffers, Longest Prefix Match (LPM), and so on. Within the context of an eBPF programme, function calls enable the definition and invocation of functions. Akin to the way in which the `execve()` system call functions for ordinary processes, tail calls [1][2][3][4] have the ability to call and run another eBPF programme while simultaneously replacing the execution environment. eBPF programmes are efficient and compact computer instructions that may be run directly within the Linux kernel. They are written in C, however a version that has restrictions is referred to as Restricted-C. This subset of C was chosen after much deliberation in order to make the environment in which eBPF programmes run as safe and effective as possible. Because it only supports a subset of eBPF’s functions and data types, it makes executing eBPF code in the kernel far less risky. The limitations placed on loops in Restricted-C and the use of floating point integers are two of the numerous constraints that are considered significant[7]. Real-time interaction with the system is made possible by the fact that eBPF programmes may be tied to a wide variety of events in the system, including system calls, network packets, and others.

Let’s have a look at a diagram that illustrates a typical workflow for the process of building and deploying an eBPF programme. The eBPF programming language is a limited form of C that uses maps. Since the Linux kernel anticipates that eBPF programmes will be loaded in bytecode, the LLVM is responsible for compiling restricted-C code into eBPF bytecode. With the help of the BPF system call, an eBPF bytecode program may be loaded into the eBPF verifier that’s included in the Linux kernel. In most cases, this is accomplished with the use of a library, the BPF Compiler Collection (BCC)[8]. After the verifier confirms that the program has not introduced any vulnerabilities and has been correctly written, it is sent on to the JIT compiler, which converts the bytecode into the native machine code [9]. After being loaded into

Table 2: Extracted top 15 features description of CIC-IDS-2017 Dataset for DT and RF

F.No	Feature	Description
1	“Idle Min”	“Minimum idle time observed in a network flow”
2	“Bwd Packet Length Min”	“Minimum length of the backward packets”
3	“Idle Mean”	“Average idle time in a network flow”
4	“Fwd IAT Total”	“Forward Inter-arrival Time Total”
5	“Bwd Packet Length Mean”	“Average length of the backward packets”
6	“Fwd IAT Mean”	“Average Inter-arrival time between consecutive forward packets”
7	“Min Packet Length”	“Smallest length observed among all the packets”
8	“Packet Length Mean”	“Average length of all the packets”
9	“Fwd IAT Max”	“Maximum Inter-arrival time between consecutive forward packets”
10	“Average Packet Size”	“Mean packet size observed in a network flow”
11	“Max Packet Length”	“Maximum length observed among all the packets”
12	“Packet Length Variance”	“Variance of packet lengths in a network flow”
13	“Avg Bwd Segment Size”	“Average size of backward segments in a network flow”
14	“Bwd Packet Length Max”	“Maximum length of the backward packets”
15	“Idle Max”	“Maximum idle time observed in a network flow”

the kernel, an eBPF program has to be associated to an event before it can be used. Whenever the event takes place, the eBPF program (or programs) that are related with it are executed. In this scenario, Sockets enables the attachment of an eBPF programme to a network interface [10]. As a result, data packets are routed via the eBPF process running in the kernel space before being sent to the real user process.

2 Related Work

[11] suggests a flow-based IDS that may be implemented using ML in eBPF. Used the widely used CIC-IDS-2017 dataset and trained the DT using sci-kit-learn using a

Table 3: Extracted top 10 features description of CIC-IDS-2017 dataset for SVM and TwinSVM

No	Feature	Description
1	“Idle Mean”	“Average idle time in a network flow”
2	“Fwd IAT Max”	“Maximum Inter-arrival time between consecutive forward packets”
3	“Packet Length Mean”	“Average length of all the packets”
4	“Idle Min”	“Minimum idle time observed in a network flow”
5	“Bwd Packet Length Max”	“Maximum length of the backward packets”
6	“Max Packet Length”	“Maximum length observed among all the packets”
7	“Avg Bwd Segment Size”	“Average size of backward segments in a network flow”
8	“Packet Length Variance”	“Variance of packet lengths in a network flow”
9	“Idle Max”	“Maximum idle time observed in a network flow”
10	“Bwd Packet Length Mean”	“Average length of the backward packets”

maximum number of leaves of one thousand and a maximum depth of ten with the training and testing ration of two to one. This results in 0.9 accuracy score on the dataset used for testing. Using the previously taught DT model, he also developed the same IDS being used in userspace as well as in the eBPF. Written code is identical in every respect, with the exception of the data structures. This is due to the fact that eBPF does not support a large number of the standard data structures. Hash maps and other eBPF data structures are not available in a standard C userspace application. As a result, they implemented the userspace version using a straightforward hash map using code taken from the Linux kernel. The author of the work implemented IDS as a traditional userspace programme in it by making advantage of eBPF and ensuring that the processes ran sequentially rather than simultaneously. Ten seconds are spent running both implementations. They evaluated the data and concluded that the userspace implementation analyses 125420 packets every second, compared to 152274 for eBPF. Because of this, eBPF is nearly 20% faster than userspace.

The paper [12] makes a suggestion for the design and implementation of an IDS that makes use of eBPF inside the Linux kernel. To begin, they suggested using a method based on eBPF to design and deploy IDS systems. They develop and execute an IDS that is comprised of two components that collaborate with one another. The initial portion of the code is executed in the Linux kernel. It does quick pattern matching with eBPF in order to pre-drop a very big part of packets that have no possibility

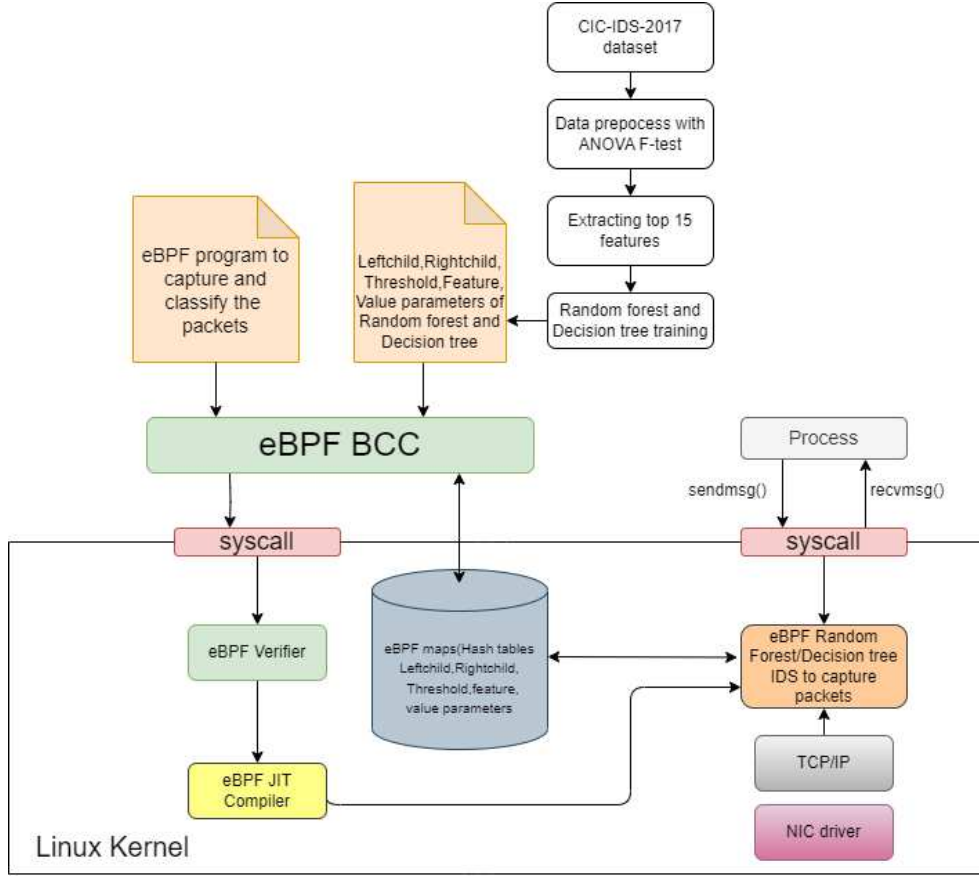


Fig. 2: Proposed model using RF and DT algorithms

of matching any rule. This is done to save bandwidth. The user's environment is the focus of the second component. It investigates the packets that were left behind by the previous portion in order to locate the rules that correspond to those packets. Under many measured conditions, an IDS system's maximum throughput may exceed Snort's by a factor of three.

Author[2] states that eBPF enables runtime modification, interaction, and kernel programmability. XDP (eXpress Data Path) framework utilizes eBPF to write programs to process packets closer to the NIC for fast packet processing. He states that programs can be written in C or P4[13] languages, compiled into eBPF instructions, and then loaded into the kernel, providing an eBPF runtime environment. His work will include eBPF and XDP rapid packet processing theory and practise. Theoretically, he covered BPF and eBPF machines and the Linux kernel's eBPF system. He demonstrated eBPF and the XDP hook with examples and tools. He thinks eBPF and XDP may advance new research initiatives since they process packets quickly.

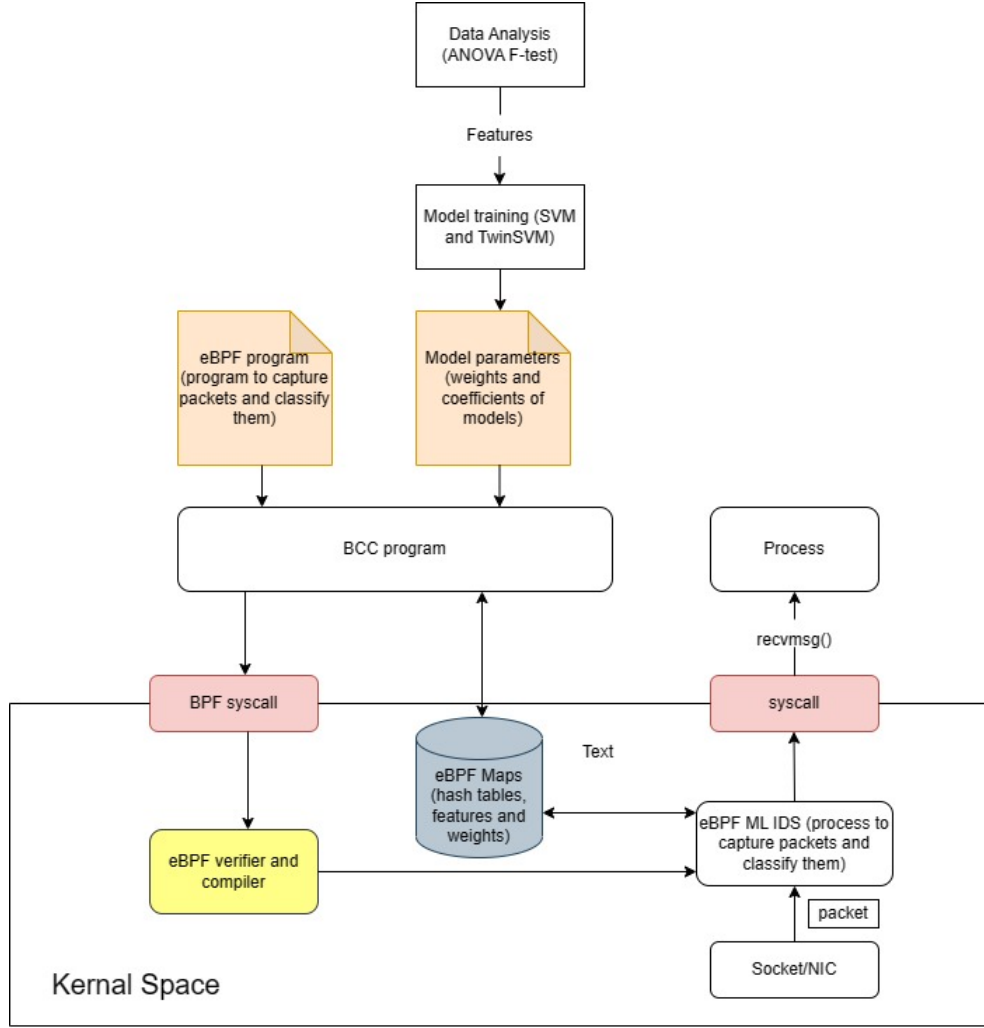


Fig. 3: Proposed model using SVM and TwinSVM algorithms

Snort [14] and Suricata [15] are both capable of filtering packets using eBPF. However, none of them are capable of using eBPF to match pattern in the packet content, and they only parse as far as the layer-3 header. Utilising the -F command-line option in Snort enables users to provide a filter expression in the way of tcpdump. After some time has passed, the phrase will be transformed into eBPF commands. Similarly, Suricata employs eBPF for XDP-based flow bypassing, load balancing, and packet filtering. The main difference between our solution and Suricata is that we support using customized eBPF scripts instead of pre-written expressions. Additionally, by employing eBPF, our system can look at packet payloads. When the context is an eBPF file, however, Suricata does not do pattern matching. DPI in Suricata utilizing

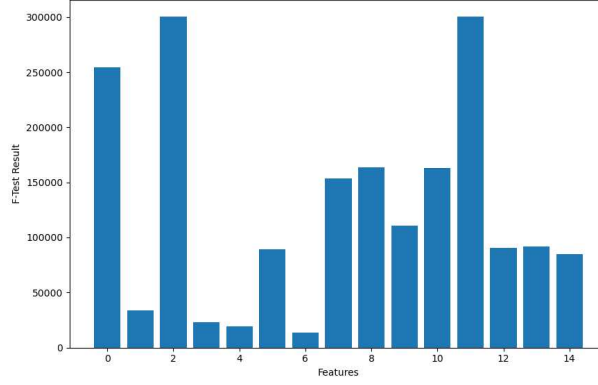


Fig. 4: ANOVA F-test on top 15 features of DoS/DDoS dataset for Random Forest and Decision Tree

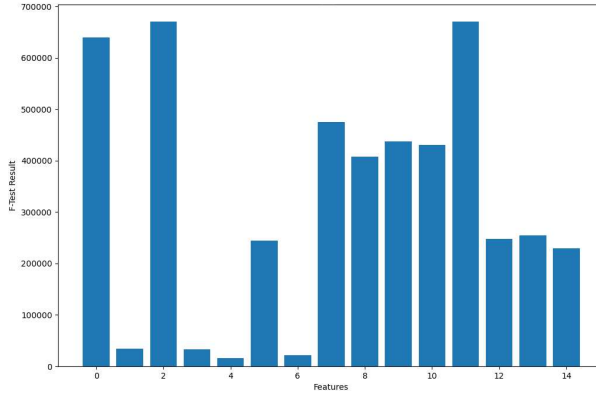


Fig. 5: ANOVA F-test on top 15 features of overall (all packets inspection) dataset for Random Forest and Decision Tree

eBPF is, therefore, no longer possible. An eBPF-based DPI method was created [16] in order to identify the different kinds of video frames that were conveyed in the packets. It can only handle packets in one format, which is inadequate for an IDS. ebpFH is an eBPF-based host-based IDS which makes use of eBPF [17]. On the other hand, it does not identify network abnormalities but rather system anomalies.

A comprehensive analysis of eBPF was carried out by the authors of [2]. This study’s results include technical specifics and a breakdown of the locations that have implemented eBPF. eBPF has been used to rewrite a significant number of the network’s fundamental functionalities, including but not limited to routing [18], switching [19], and firewalls [20]. These features include load balancing [21], key-value storage [22], application level filtering [23], and DDoS mitigation [24]. In-KeV [25] is a framework for developing network services that run within the kernel. eBPF creates in-kernel

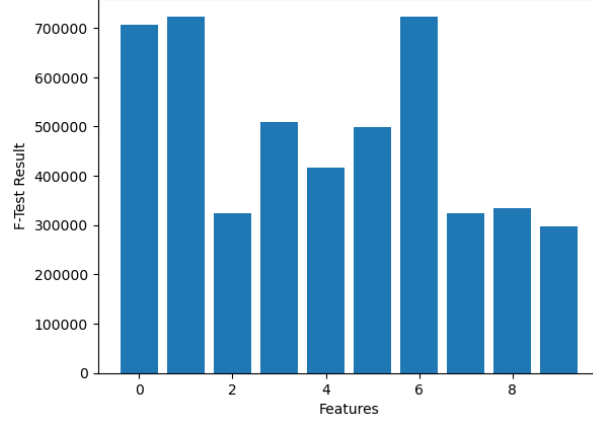


Fig. 6: ANOVA F-test on top 10 features of overall (all packets inspection) dataset for SVM and TwinSVM

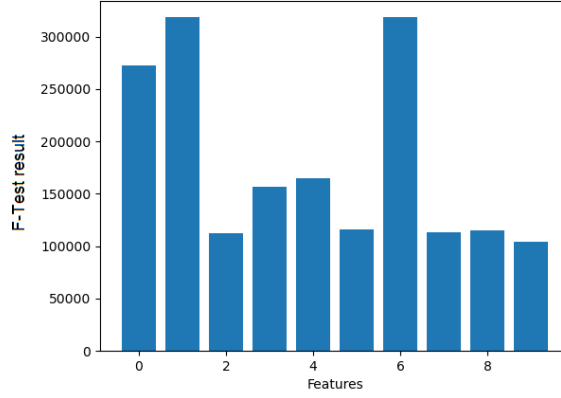


Fig. 7: ANOVA F-test on top 10 features of DoS/DDoS dataset for SVM and TwinSVM

Service Function Chains (SFC) via tail calls. [26] explored the limits of eBPF and their own experiences with eBPF. The authors examined the performance of eBPF to filter packets based on the 5-tuple information in the packet header in [27]. Using eBPF, [28] created an open-source 5G mobile gateway. 5G networks are becoming more commonplace. eBPF was utilized for monitoring the communications between Virtual Machines (InterVM) by the creators of [29]. A framework for implementing eBPF-based network functions for microservices [30].

When implementing complicated network functions, the authors of [26] pointed out several significant parameters that impact the performance of eBPF. This

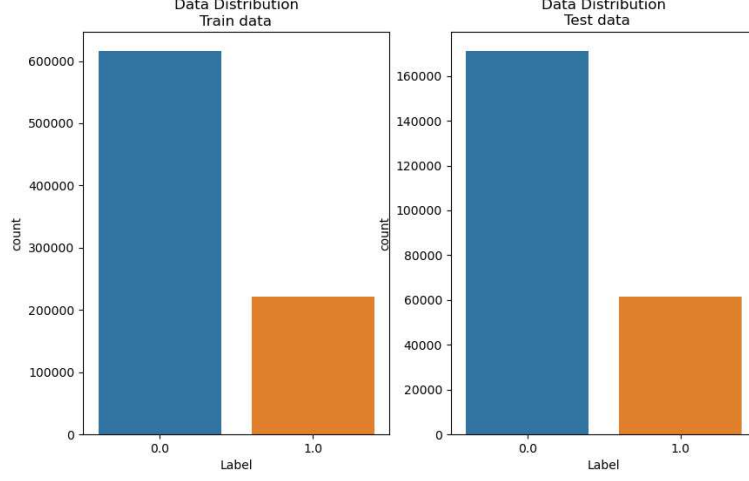


Fig. 8: Data distribution of top 15 features in CIC-IDS-2017 dataset with Label 0 is Benign and Label 1 is Attacks.

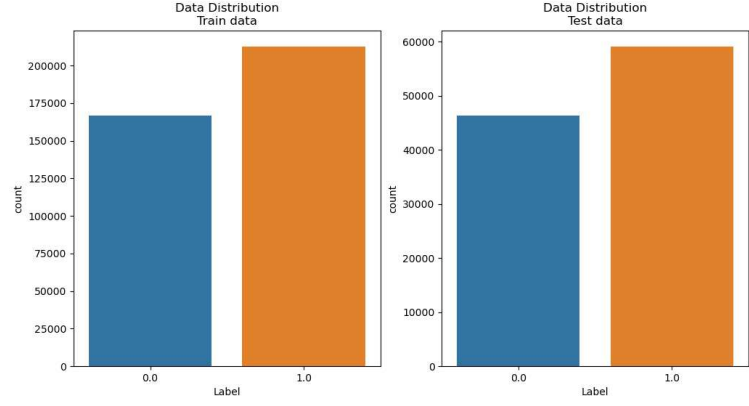


Fig. 9: Data distribution of top 15 features for DoS/DDoS in CIC-IDS-2017 dataset with Label 0 is Benign and Label 1 is Attacks.

article [27] focused on eBPF-based firewalls, which are simpler than IDSs. cite-hohlfeld2019demystifying examined XDP performance in VM and hardware offloading situations [31]. The results of this study demonstrated that XDP within VMs saw performance reductions. String matching is handled by Snort using a combination of the Boyer–Moore (BM) [32] and Aho–Corasick (AC) [33] algorithms. [34] suggests using AI to identify performance irregularities using eBPF. eBPF is used in [35], [36] and also [37] in order to build countermeasures for DoS assaults. They don't make use of ML at all.

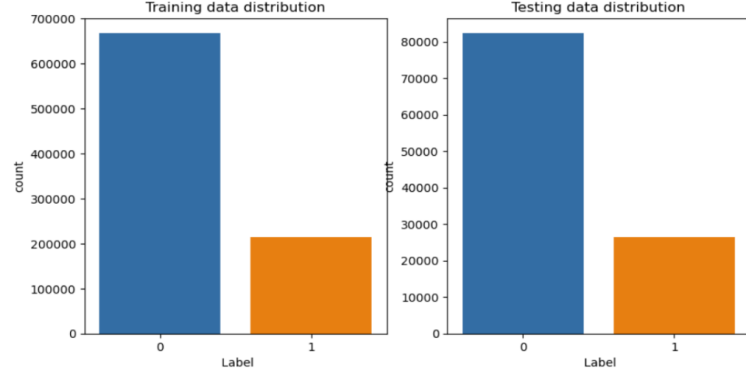


Fig. 10: Data distribution of top 10 features in CIC-IDS-2017 dataset with Label 0 is Benign and Label 1 is Attacks.

3 Proposed Method and Experimental results

The steps involved in this process are:

- We are making use of data from the popular CIC-IDS-2017 dataset [38].
- Once the raw data is obtained, it undergoes pre-processing, which includes data transmission (reading data from files), cleaning (infinity, NULL values, etc.), and reduction (size)..
- Following the pre-processing step, to extract specific features from the pre-processed data by using the ANOVA F-test method.
- The extracted features are analyzed for intrusion detection using ML algorithms.
 - After training and testing with RF and DT, export the model parameters, including the left child, right child, threshold, features, and value of each decision tree in the RF.
 - After training and testing with SVM and TwinSVM, export the model parameters, including coefficients, intercepts, and features.
- Lastly, write an eBPF program that captures network traffic and utilizes trained model parameters to detect attacks within the kernel

3.1 Testbed setup

The required system specifications are “Ubuntu OS 22.04 operating system with 4 GB RAM and Intel(R) Core(TM) i5-8250U CPU @ 1.60GHz processor”. The C program was written and compiled using GCC (GNU Compiler Collection) version 11.3.0. The experiments were conducted using Python 3.10.6.

3.2 Feature extraction

We are using the CIC-IDS-2017 dataset and given description in Table 1, a network traffic dataset containing labeled network traffic data, including benign traffic and

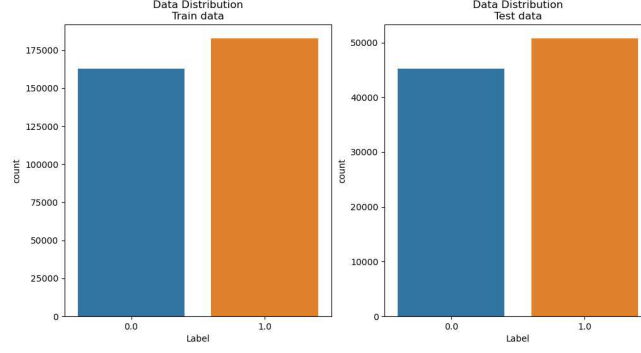


Fig. 11: Data distribution of top 10 features for DoS/DDoS in CIC-IDS-2017 dataset with Label 0 is Benign and Label 1 is Attacks.

various types of attacks. The dataset is designed to be used for intrusion detection and prevention purposes. By using this dataset, we can detect DoS/DDoS attacks. First, we should analyze the CIC-IDS-2017 dataset to get some essential features that can be trained into machine learning algorithms. Training is done by using the Analysis of Variance (ANOVA) F-test technique. The CIC-IDS-2017 dataset is a collection of network traffic data consisting of 78 features. These features are destination port, total forward packets, total backward packets, minimum packet length, etc. This dataset is commonly used for testing intrusion detection systems. In previous work, the author [11] used 12 features of the overall packet inspection of CIC-IDS-2017 dataset for the DT model, which is used in eBPF. So, we worked on the top 15 features of the comprehensive packet inspection of CIC-IDS-2017 dataset and the DoS/DDoS attack of the CIC-IDS-2017 dataset for better performance for DT, RF, SVM, and Twin-SVM training. But for SVM and Twin-SVM, it takes more time and consuming the entire RAM memory for 15 feature training, so we reduced it to 10 features for training and testing. To detect DoS/DDoS attacks using the CIC-IDS-2017 dataset, we first analyze the dataset; then, by using ANOVA F-test, we extract the top 15 features from 78 features for DT and RF and the top 10 features from 78 features for SVM and TwinSVM. ANOVA F-test determines features that have the most significant impact on the target variable. ANOVA is a statistical method that compares the means of multiple groups to determine if there are substantial differences among them. The selection process involves calculating the F-value for each feature using ANOVA. The F-value represents the ratio of the variation between groups to the variation within groups if the features with high F-values indicate strong dependence on the target variable. By using ANOVA F-test feature selection, we can reduce the dimensionality of the dataset.

Based on several considerations, we are using the top 15 or 10 features, extracted using an ANOVA F-test, instead of all features from the CIC-IDS-2017 dataset.

1. Dimensionality reduction: The CIC-IDS-2017 dataset contains 78 features; using too many features sometimes leads to overfitting, which will reduce the model's accuracy.
2. Feature relevance: All features in the dataset may not be equally informative or

relevant for data analysis. Using the ANOVA F-test helps identify features that have a more significant impact on the target variable. 3. Complexity and resource constraints: Including all 78 features in the eBPF implementation might increase the complexity of the code. eBPF programs are typically designed to run efficiently within limited resource constraints. Using more features could lead to increased memory usage, longer processing times, and performance issues. However, there are some drawbacks to using only the top 15 or top 10 features because the model might only detect the attack if the features of malicious packets are represented in the top 15 or top 10 features.

Algorithm 1 Proposed Decision Tree Algorithm

```

1: Initialize children_left, children_right, threshold, feature and value from model
   parameters
2: Initialize real_feature_value from packet data
3: current_node  $\leftarrow$  0
4: for  $i = 0$  to  $MAX\_TREE\_DEPTH - 1$  do
5:   current_left_child  $\leftarrow$  children_left[current_node]
6:   current_right_child  $\leftarrow$  children_right[current_node]
7:   current_feature  $\leftarrow$  feature[current_node]
8:   current_threshold  $\leftarrow$  threshold[current_node]
9:   if current_left_child = TREE_LEAF || current_right_child = TREE_LEAF
   then
10:    break
11:   else
12:     real_feature_value  $\leftarrow$  [current_feature]
13:     if real_feature_value  $\leq$  current_threshold then
14:       current_node  $\leftarrow$  current_left_child
15:     else
16:       current_node  $\leftarrow$  current_right_child
17:     end if
18:   end if
19: end for
20: correct_value = value[current_node]
21: prediction  $\leftarrow$  1 if correct_value=1 else 0

```

By using ANOVA F-test technique on the CIC-IDS-2017 dataset, we extracted the top 15 features for Random Forest and Decision Tree algorithms is shown in Table 2. The ANOVA test on top 15 features on CIC-IDS-2017 dataset is shown in Figure 5 and on CIC-IDS-2017 dataset with DoS/DDoS is shown in Figure 4. We extracted the top 10 features for SVM and TwinSVM algorithms is shown in Table 3. The ANOVA test on top 10 features on CIC-IDS-2017 dataset shown in Figure 6 and CIC-IDS-2017 dataset with DDoS/DoS is shown in Figure 7.

Algorithm 2 Proposed Random Forest Algorithm

```
1: Initialize children_left, children_right, threshold, feature and value from model
   parameters
2: Initialize real_feature_value from packet data
3: Initialize tree_number from model parameters
4: current_node  $\leftarrow$  0
5: True_count  $\leftarrow$  0
6: False_count  $\leftarrow$  0
7: for tree_number = 0 to n_estimators - 1 do
8:   for i = 0 to MAX_TREE_DEPTH - 1 do
9:     current_left_child  $\leftarrow$  children_left[current_node, tree_number]
10:    current_right_child  $\leftarrow$  children_right[current_node, tree_number]
11:    current_feature  $\leftarrow$  feature[current_node, tree_number]
12:    current_threshold  $\leftarrow$  threshold[current_node, tree_number]
13:    if current_left_child = TREE_LEAF || current_right_child =
       TREE_LEAF then
14:      break
15:    else
16:      real_feature_value  $\leftarrow$  [current_feature, tree_number]
17:      if real_feature_value  $\leq$  current_threshold then
18:        current_node  $\leftarrow$  current_left_child
19:      else
20:        current_node  $\leftarrow$  current_right_child
21:      end if
22:    end if
23:  end for
24: end for
25: for tree_number = 0 to n_estimators - 1 do
26:   correct_value = value[current_node, tree_number]
27:   if *correct_value then
28:     True_count  $\leftarrow$  True_count + 1
29:   else
30:     False_count  $\leftarrow$  False_count + 1
31:   end if
32: end for
33: if True_count > False_count then
34:   correct_value = 1
35: else
36:   correct_value = 0
37: end if
38: prediction  $\leftarrow$  1 if correct_value=1 else 0
```

3.3 Decision Tree

We make use of the well-known CIC-IDS-2017 dataset with the top 15 features for data analysis and prediction, which can be found at [38]. We train the DT using scikit-learn, with a maximum depth of fifteen and a maximum number of leaves of one thousand, using a train/test split of eighty percent and twenty percent, respectively. Data distribution with Label 0 is “Benign” and with Label 1 is “Attack” is shown in Figure 8 and Data distribution for DOS or DDoS with Label 0 is “Benign” and with Label 1 is “Attack” is shown in Figure 9. After training and testing with the DT, we export model parameters left children, right children, threshold, features, and value of each DT. The proposed DT algorithm is given in Algorithm 1. The overall (all packets inspection) dataset and DoS/DDoS performance parameters are given in Table 4, Table 5.

Algorithm 3 Proposed SVM Algorithm

```
1: Initialize coefficients from model parameters
2: Initialize intercept from model parameters
3: Initialize features from packet data
4: sum  $\leftarrow$  0
5: for  $i \leftarrow 0$  to  $MAX\_FEATURES - 1$  do
6:   sum  $\leftarrow$  sum + coefficients[i] * features[i]
7: end for
8: sum  $\leftarrow$  sum + intercept
9: prediction  $\leftarrow$  1 if sum  $\geq$  0 else 0
```

3.4 Decision Tree in eBPF

After getting the parameters of the DT model, we write an eBPF program to process the incoming malicious packets. In this eBPF program, we created eBPF maps to store the parameters of the DT model like left children, right children, threshold, features, value, etc. The eBPF program code is loaded and combined with other necessary code, such as maps and tables. eBPF maps store left and right children, thresholds, features, and value parameters and can access data within the eBPF program. The eBPF program, along with its associated maps, is loaded into the kernel. The program is attached to a specific hook or event point within the kernel, socket. The socket where it sends and receives packets from network interfaces. Each captured packet is passed to the eBPF program, which is attached to the socket for packet filtering shown in Figure 2. Time taken in user space Vs. Kernel space of overall (all packets inspection) and DoS/DDoS detection Mean time for packet/s is shown in Table 6 and Table 7.

3.5 Random Forest model

We implemented the RF model on the CIC-IDS-2017 dataset with the top 15 features for data analysis and prediction. Data distribution with Label 0 is “Benign” and with

Label 1 is “Attack” is shown in Figure 8 and Data distribution for DOS or DDOS with Label 0 is “Benign” and with Label 1 is “Attack” is shown in Figure 9. After training and testing with RF, we export model parameters: left children, right children, threshold, features, and value of each DT. The proposed RF algorithm is given in Algorithm 2. The overall (all packets inspection) dataset and DoS/DDoS detection performance parameters are shown in Table 4 and Table 5.

Algorithm 4 Proposed TwinSVM Algorithm

```

1:  $sum1 \leftarrow 0$ 
2:  $sum2 \leftarrow 0$ 
3: Initialize  $coefficients1$  from model parameters
4: Initialize  $coefficients2$  from model parameters
5: Initialize  $intercept1$  from model parameters
6: Initialize  $intercept2$  from model parameters
7: Initialize  $w1mod$  from model parameters
8: Initialize  $w2mod$  from model parameters
9: Initialize  $features$  from packet data
10: for  $i \leftarrow 0$  to  $MAX\_FEATURES - 1$  do
11:    $sum1 \leftarrow sum1 + coefficients1[i] * features[i]$ 
12:    $sum2 \leftarrow sum2 + coefficients2[i] * features[i]$ 
13: end for
14:  $sum1 \leftarrow sum1 + intercept1$ 
15:  $sum2 \leftarrow sum2 + intercept2$ 
16:  $prediction \leftarrow 1$  if  $sum1 * w2mod.val \geq sum2 * w1mod.val$  else 0

```

3.6 Random Forest algorithm in eBPF

After getting the parameters of the RF model, we write an eBPF program to process the incoming malicious packets. In this eBPF program, we created eBPF maps to store the parameters of the RF model like Left Children, Right Children, Threshold, Features, Value of each DT, etc. The eBPF program code is loaded and combined with other necessary code, such as maps and tables. eBPF maps store left and right children, thresholds, features, and value parameters and can access data within the eBPF program. The eBPF program, along with its associated maps, is loaded into the kernel. The program is attached to a specific hook or event point within the kernel, such as a socket. The socket where it sends and receives packets from network interfaces. Each captured packet is passed to the eBPF program, which is attached to the socket for packet filtering shown in Figure 2. Time taken in user space Vs. Kernel space of overall (all packets inspection) and DoS/DDoS detection Mean time for packet/s is shown in Table 6 and Table 7.

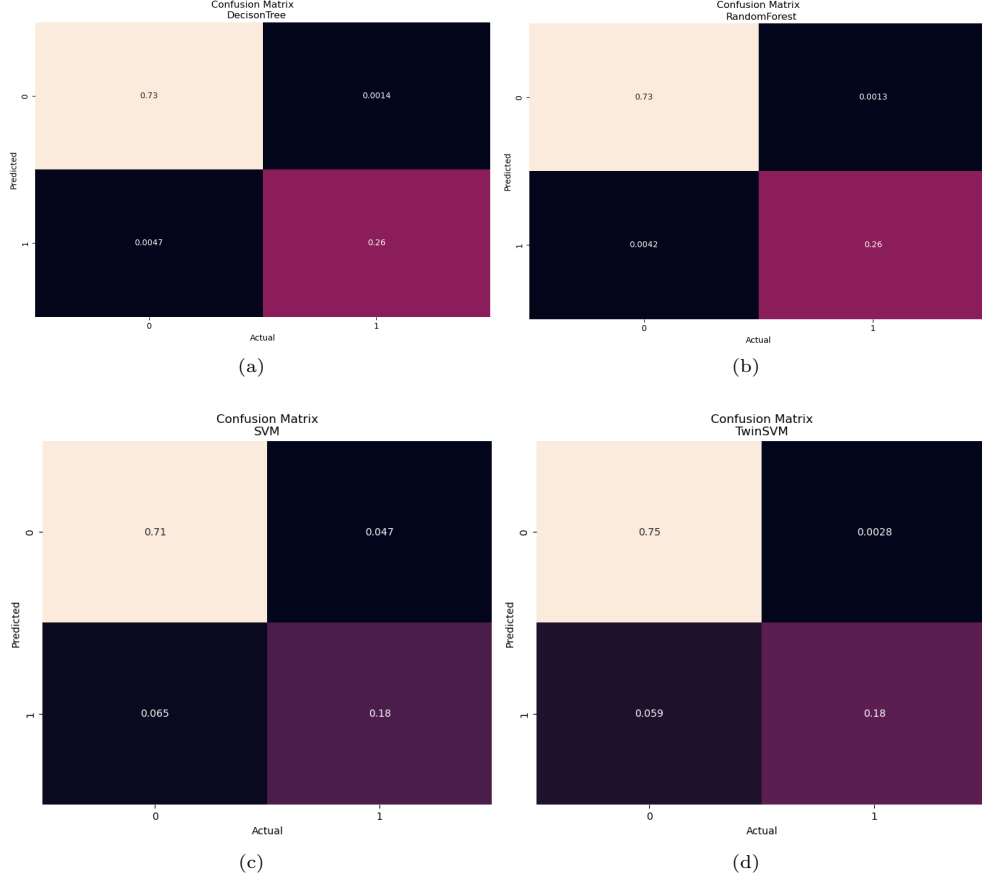


Fig. 12: Confusion Matrix for (a)Decision Tree , (b)Random forest , (c)SVM , and (d)TwinSVM of overall (all packets inspection) CIC-IDS-2017 dataset.

3.7 SVM and TwinSVM

We trained machine learning algorithms, SVM and TwinSVM in Python on the CIC-IDS-2017 dataset with top 10 features. Data distribution with Label 0 is “Benign” and with Label 1 is “Attack” is shown in Figure 10 and Data distribution for DOS/ DDOS with Label 0 is “Benign” and with Label 1 is “Attack” is shown in Figure 11. Then, we deploy the models in eBPF and userspace to compare the performance regarding the number of packets processed in a timeframe. One of the main hurdles in deploying these algorithms was representing the weights, which are floating point numbers in eBPF. eBPF does not allow floating-point numbers, so instead, we use a fixed-point representation of these floating-point numbers, which is nothing but multiplying all the weights with some large values (in our case, 216). This method has no problem for linear SVM and TwinSVM since they compare distances using these weights. However, Neural Networks (NN) and XGBoost inherently operate continuous values for outputs

and later convert them into classes using some function like sigmoid or tanh. The final output is some function of the weights with some non-linear transformation, so we cannot use fixed point representation in NN and XGBoost. The proposed SVM and TwinSVM algorithms are given in Algorithm 3 and Algorithm 4. The overall (all packets inspection) dataset and DoS/DDoS detection performance parameters are shown in Table 4 and Table 5.

3.8 SVM and TwinSVM in eBPF

We implemented SVM and TwinSVM with a different kind of training, wherein we split the dataset into batches of 100 and trained a separate model for each of the 100 data points. Then we pick the best models of these and take a mean of their weights. We assign these weights to a new model, and it outperforms the top models by a slight margin over the validation set. Time taken in user space Vs. Kernel space of overall (all packets inspection) and DoS/DDoS detection Mean time for packet/s is shown in Table 6 and Table 7.

4 Performance Analysis

The performance parameters of ML algorithms are accuracy, precision, recall/sensitivity, F1-score, and specificity. Using the widely used CIC-IDS-2017 dataset with 12 features, Maximilian Bachl et al. [11] train the DT using sci-kit-learn with a maximum depth of ten and a maximum number of leaves of one thousand using a train/test split of 2:1. This results in an accuracy of 99.0% on the testing dataset after training. We used the same CIC-IDS-2017 dataset and extracted the top 15 features using the ANOVA F-test method. We train the DT using sci-kit-learn with a maximum depth of 15, and a train/test split of 4:1. The comparison of accuracy in user space and kernel space is shown in Table 8. We can see in this table, our experimental results show an accuracy of 99.38 percent when testing the dataset after training, which is better than the accuracy attained with [11] (existing related work). Further, compared to DT and SVM models, RF is performing better, this can be seen with the accuracy of 99.44 percent. Also, we can see in the table that we calculated DoS/DDoS detection, and the accuracy was found to be 99.57 percent.

RF is an ensemble learning method for classification and regression that can detect DoS/DDoS attacks by analyzing network traffic patterns and identifying unusual behavior that may indicate an attack. We trained RF using popular CIC-IDS-2017 with a test/train split of 1:4 with a max depth of 20. After training and testing, we export RF model parameters. This approach uses eBPF and Socket filters for DoS/DDoS mitigation. It is implemented using an RF classifier in eBPF, which helps classify incoming network packets in real time. The eBPF program attached to the socket hook can filter and record the necessary traffic features and then pass the data to user space. The algorithm then makes the prediction, and then eBPF can take the required actions, such as dropping the packets or redirecting them to a DDoS mitigation system. In this process, we achieve better performance using RF compared to DT, SVM, and TwinSVM, 99.44 overall (all packets inspection) accuracy and 99.58 DoS/DDoS detection accuracy.

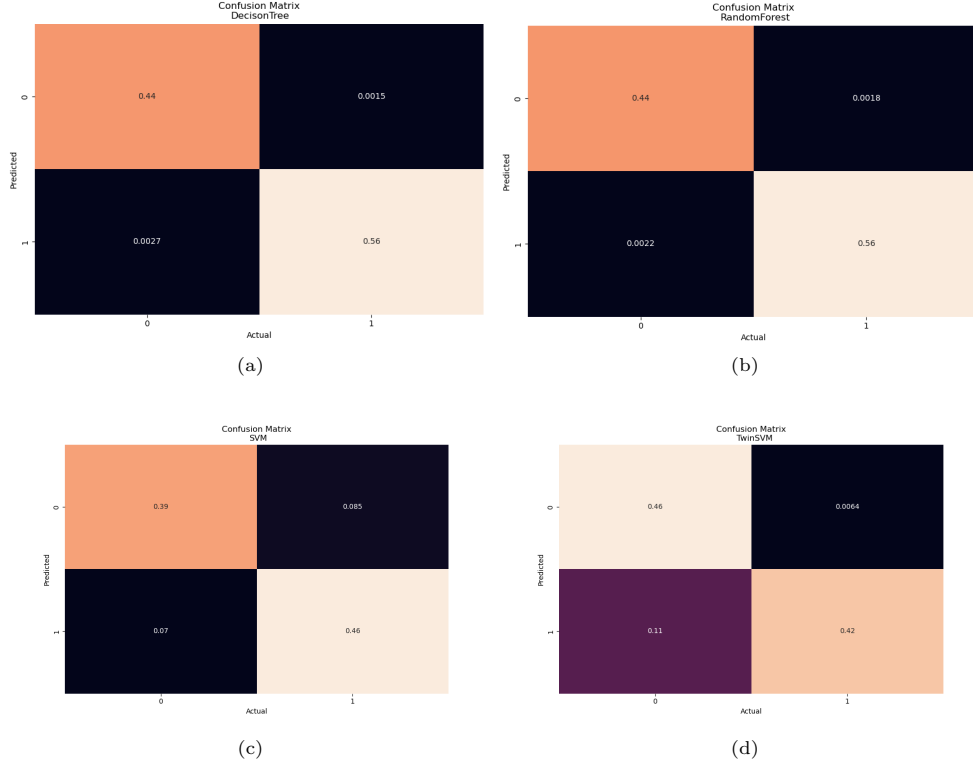


Fig. 13: Confusion Matrix for (a)Decision Tree , (b)Random forest , (c)SVM , and (d)TwinSVM of DoS/DDoS in CIC-IDS-2017 dataset.

DT is a supervised machine learning algorithm used for classification and regression purposes. It is a tree-based structure where each node represents a feature or attribute, each edge means a decision rule, and each leaf node represents prediction. We trained DT using popular CIC-IDS-2017 with a test/train split of 1:4 with a max depth of 15. After training and testing, we export DT model parameters. In this method, we use DT in eBPF, which is attached to a specific event named Socket, which will filter network data and pass it to the user space. In this method, DT model parameters predict each packet, and then eBPF takes necessary action, such as dropping or redirecting packets. In this process, we achieve more performance compared to DT implemented by this author[11] with 99.38 accuracies for the overall (all packets inspection) CIC-IDS-2017 dataset and 99.57 DoS/DDoS accuracy.

SVM and TwinSVM algorithms have accuracy for the overall (all packets inspection) dataset of 88.74 and 93.82. Comprehensive dataset using different ML algorithms confusion matrix results are shown in Figure 12, and DoS/DDoS confusion matrix is shown in Figure 13. Both a traditional userspace programme version and eBPF version are implemented for the execution of the intrusion detection system (IDS). Both programmes were executed independently for ten seconds. According to the data in

Table 4: Experimental results of overall (all packets inspection) CIC-IDS-2017 dataset in userspace

Performance Parameters	Method	DT	RF	SVM	TwinSVM
Accuracy	Train	99.52	99.59	88.77	93.87
	Test	99.38	99.44	88.74	93.82
Precision	Train	99.71	99.74	78.97	98.58
	Test	99.46	99.51	78.97	98.49
Recall/ Sensitivity	Train	98.49	98.72	73.41	75.9
	Test	98.21	98.37	73.26	75.78
F1 Score	Train	99.09	99.23	76.09	85.76
	Test	98.83	98.94	76.01	85.65
Specificity	Train	99.89	99.91	93.71	99.65
	Test	99.81	99.82	93.73	99.63

Table 5: Experimental results of DoS/DDoS of CIC-IDS-2017 dataset in userspace

Performance Parameters	Method	DT	RF	SVM	TwinSVM
Accuracy	Train	99.73	99.82	84.43	88.49
	Test	99.57	99.58	84.43	88.42
Precision	Train	99.87	99.88	84.28	98.53
	Test	99.71	99.65	84.31	98.49
Recall/ Sensitivity	Train	99.65	99.80	86.74	79.42
	Test	99.52	99.60	86.70	79.33
F1 Score	Train	99.76	99.84	85.49	87.95
	Test	99.62	99.62	85.48	87.88
Specificity	Train	99.83	99.85	81.83	98.67
	Test	99.64	99.55	81.88	98.63

Tables 6 and 7, the userspace implementation examines a lower number of packets per second than the eBPF version does.

5 Declarations

Not Applicable

6 Conclusion and Future Research

eBPF is a bytecode-based virtual machine that runs programs without modifying the kernel source code. The eBPF implementation is faster than the userspace. Used CIC-IDS-2017 dataset and trained with DT, RF, SVM and TwinSVM using sci-kit-learn with test/train split of 1:4. Our experimental results show that the accuracy of our proposed ML algorithms, DT, RF, SVM, and TwinSVM, outperforms the existing

Table 6: Time taken in user space vs kernel space of overall (all packets inspection) CIC-IDS-2017 dataset

Algorithms	Userspace (Mean)	eBPF (Mean)
Decision Tree packet/s	46239	109691
Random Forest packet/s	45978	108534
SVM packet/s	45590	92978
TwinSVM packet/s	38430	109865

Table 7: Time taken in user space vs kernel space of DoS/DDoS of CIC-IDS-2017 dataset

Algorithms	Userspace (Mean)	eBPF (Mean)
Decision Tree packet/s	42463	106421
Random Forest packet/s	41632	105245
SVM packet/s	49376	92581
TwinSVM packet/s	42487	117536

Table 8: Accuracy, user space, eBPF space comparison with related work

Author/Parameter	Algorithm	Accuracy	User Space packet/s	eBPF packet/s
[11]	Decision Tree	99	125420	152274
Proposed	Decision Tree	99.38	46239	109691
Proposed	Random Forest	99.44	45978	108534
Proposed	SVM	88.74	45590	92978
Proposed	TwinSVM	93.82	38430	109865

related work: 99.38, 99.44, 88.73, and 93.82, respectively. For future research analysis, we can build an IDS that utilizes the eBPF and the XDP with ML algorithms such as DT, RF, SVM and TwinSVM. XDP is a programmable data path interface in the Linux kernel that allows for efficient filtering and manipulation of network packets at the Network Interface Card (NIC) driver level. XDP programs are written in eBPF bytecode, are attached to network devices, and can execute on the NIC before the packet reaches the kernel network stack and act on the packet directly on the NIC.

References

- [1] LLC, M.: eBPF Documentation. <https://ebpf.io/what-is-ebpf/> Accessed 2023-06-19
- [2] Vieira, M.A., Castanho, M.S., Pacífico, R.D., Santos, E.R., Júnior, E.P.C., Vieira, L.F.: Fast packet processing with ebpf and xdp: Concepts, code, challenges, and applications. *ACM Computing Surveys (CSUR)* **53**(1), 1–36 (2020)
- [3] Høiland-Jørgensen, T., Brouer, J.D., Borkmann, D., Fastabend, J., Herbert, T.,

- Ahern, D., Miller, D.: The express data path: Fast programmable packet processing in the operating system kernel. In: Proceedings of the 14th International Conference on Emerging Networking Experiments and Technologies, pp. 54–66 (2018)
- [4] Sharaf, H., Ahmad, I., Dimitriou, T.: Extended berkeley packet filter: An application perspective. *IEEE Access* (2022)
 - [5] Lattner, C., Adev, V.: The llvm compiler framework and infrastructure tutorial, 15–16 (2005). Springer
 - [6] Dumazet, E.: A JIT for packet filters (2011). <https://lwn.net/Articles/437981/>
 - [7] Rybczynska, M.: Bounded loops in bpf for the 5.3 kernel (2019)
 - [8] Project, I.: BCC (BPF Compiler Collection) (2022). <https://github.com/iovisor/bcc>
 - [9] Miller, D.: BPF Verifier Overview (2019). <https://lwn.net/Articles/794934/>
 - [10] Maguire, A.: Notes on BPF (1)—A Tour of Program Types (2019)
 - [11] Bachl, M., Fabini, J., Zseby, T.: A flow-based ids using machine learning in ebpf. arXiv preprint arXiv:2102.09980 (2021)
 - [12] Wang, S.-Y., Chang, J.-C.: Design and implementation of an intrusion detection system by using extended bpf in the linux kernel. *Journal of Network and Computer Applications* **198**, 103283 (2022)
 - [13] Bosshart, P., Daly, D., Gibb, G., Izzard, M., McKeown, N., Rexford, J., Schlesinger, C., Talayco, D., Vahdat, A., Varghese, G., *et al.*: P4: Programming protocol-independent packet processors. *ACM SIGCOMM Computer Communication Review* **44**(3), 87–95 (2014)
 - [14] Roesch, M.: Snort users manual. <http://www.snort.org> (2002)
 - [15] Leblond, É., Manev, P.: Introduction to eBPF and XDP support in Suricata (2019)
 - [16] Baidya, S., Chen, Y., Levorato, M.: ebpf-based content and computation-aware communication for real-time edge computing. In: IEEE INFOCOM 2018-IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS), pp. 865–870 (2018). IEEE
 - [17] Findlay, W.: Extended berkeley packet filter for intrusion detection implementations. PhD thesis, Honours Thesis Proposal, Carleton University (2019)
 - [18] Xhonneux, M., Duchene, F., Bonaventure, O.: Leveraging ebpf for programmable

- network functions with ipv6 segment routing. In: Proceedings of the 14th International Conference on Emerging Networking EXperiments and Technologies, pp. 67–72 (2018)
- [19] Viljoen, N., Kicinski, J.: Using ebpf as an abstraction for switching. URL http://vger.kernel.org/lpc_net2018_talks/eBPF_For_Switches.pdf (2018)
 - [20] Miano, S., Bertrone, M., Risso, F., Bernal, M.V., Lu, Y., Pi, J.: Securing linux with a faster and scalable iptables. *ACM SIGCOMM Computer Communication Review* **49**(3), 2–17 (2019)
 - [21] katra: katra (2021). <https://github.com/facebookincubator/katra>
 - [22] Lazri, K., Blin, A., Sopena, J., Muller, G.: Toward an in-kernel high performance key-value store implementation. In: 2019 38th Symposium on Reliable Distributed Systems (SRDS), pp. 268–2680 (2019). IEEE
 - [23] cilium: eBPF-based Networking, Observability, Security (2022). <https://cilium.io/>
 - [24] Miano, S., Doriguzzi-Corin, R., Risso, F., Siracusa, D., Sommesse, R., CREATE-NET, F.B.K.: High-performance server-based ddos mitigation through programmable data planes
 - [25] Ahmed, Z., Alizai, M.H., Syed, A.A.: Inkev: In-kernel distributed network virtualization for dcn. *ACM SIGCOMM Computer Communication Review* **46**(3), 1–6 (2018)
 - [26] Miano, S., Bertrone, M., Risso, F., Tumolo, M., Bernal, M.V.: Creating complex network services with ebpf: Experience and lessons learned. In: 2018 IEEE 19th International Conference on High Performance Switching and Routing (HPSR), pp. 1–8 (2018). IEEE
 - [27] Scholz, D., Raumer, D., Emmerich, P., Kurtz, A., Lesiak, K., Carle, G.: Performance implications of packet filtering with linux ebpf. In: 2018 30th International Teletraffic Congress (ITC 30), vol. 1, pp. 209–217 (2018). IEEE
 - [28] Parola, F., Risso, F., Miano, S.: Providing telco-oriented network services with ebpf: the case for a 5g mobile gateway. In: 2021 IEEE 7th International Conference on Network Softwarization (NetSoft), pp. 221–225 (2021). IEEE
 - [29] Hong, J., Jeong, S., Yoo, J.-H., Hong, J.W.-K.: Design and implementation of ebpf-based virtual tap for inter-vm traffic monitoring. In: 2018 14th International Conference on Network and Service Management (CNSM), pp. 402–407 (2018). IEEE
 - [30] Miano, S., Risso, F., Bernal, M.V., Bertrone, M., Lu, Y.: A framework for ebpf-based network functions in an era of microservices. *IEEE Transactions on Network*

- and Service Management **18**(1), 133–151 (2021)
- [31] Hohlfeld, O., Krude, J., Reelfs, J.H., R  th, J., Wehrle, K.: Demystifying the performance of xdp bpf. In: 2019 IEEE Conference on Network Softwarization (NetSoft), pp. 208–212 (2019). IEEE
 - [32] Boyer, R.S., Moore, J.S.: A fast string searching algorithm. *Communications of the ACM* **20**(10), 762–772 (1977)
 - [33] Aho, A., Corasick, M.: Efficient string matching. *Comm. ACM* **18**(6), 333
 - [34] Ben-Yair, I., Rogovoy, P., Zaidenberg, N.: Ai & ebpf based performance anomaly detection system. In: Proceedings of the 12th ACM International Conference on Systems and Storage, pp. 180–180 (2019)
 - [35] Demoulin, H.M., Pedisich, I., Vasilakis, N., Liu, V., Loo, B.T., Phan, L.T.X.: Detecting asymmetric application-layer denial-of-service attacks in-flight with finelame. In: USENIX Annual Technical Conference, pp. 693–708 (2019)
 - [36] Wieren, H.: Signature-based ddos attack mitigation: Automated generating rules for extended berkeley packet filter and express data path. Master’s thesis, University of Twente (2019)
 - [37] Choe, Y., Shin, J.-S., Lee, S., Kim, J.: ebpf/xdp based network traffic visualization and dos mitigation for intelligent service protection. In: Advances in Internet, Data and Web Technologies: The 8th International Conference on Emerging Internet, Data and Web Technologies (EIDWT-2020), pp. 458–468 (2020). Springer
 - [38] Panigrahi, R., Borah, S.: A detailed analysis of cecids2017 dataset for designing intrusion detection systems. *International Journal of Engineering & Technology* **7**(3.24), 479–482 (2018)